

Модуль 5. Docker Compose: поднимаем целую систему одной командой

5.1. От одного контейнера к экосистеме: зачем нужен Compose

5.1.1. Проблема: ручной запуск десятков команд

В предыдущих модулях мы запускали **один** контейнер. Но настоящее приложение — это не один ящик, а целый город из ящиков, которые должны работать вместе.

Представьте, что вы открываете интернет-магазин. Вам нужно:

- **API** (Python/FastAPI) — принимает заказы от пользователей.
- **База данных** (PostgreSQL) — хранит заказы, пользователей, товары.
- **Брокер сообщений** (Redis) — ставит тяжёлые задачи в очередь.

Если запускать это вручную, вам придётся выполнить примерно такой набор команд:

```
In [1]: # 1. Создать сеть, чтобы контейнеры видели друг друга
        docker network create my_network

# 2. Запустить PostgreSQL
docker run -d --name db \
    --network my_network \
    -e POSTGRES_USER=admin \
    -e POSTGRES_PASSWORD=secret \
    -e POSTGRES_DB=shop \
    -v pg_data:/var/lib/postgresql/data \
    postgres:15

# 3. Запустить Redis
docker run -d --name redis \
    --network my_network \
    -v redis_data:/data \
    redis:7

# 4. Собрать образ API
docker build -t my_api .

# 5. Запустить API
docker run -d --name api \
    --network my_network \
    -p 8000:8000 \
    -e DATABASE_URL=postgresql://admin:secret@db:5432/shop \
```

```
-e REDIS_URL=redis://redis:6379 \
my_api
```

Что здесь не так?

- **Сложно запомнить.** Пять команд, десятки флагов. Ошибешься в одном символе — ничего не работает.
- **Сложно передать коллеге.** «Напиши вот эти команды, только не перепутай порядок».
- **Сложно остановить.** Нужно останавливать каждый контейнер отдельно, не забыть удалить сеть.
- **Сложно масштабировать.** Хотите добавить второй API? Копируйте команду, меняйте порты, не запутайтесь.

5.1.2. Решение: Docker Compose

Docker Compose — это инструмент, который позволяет описать **всю систему** в одном YAML-файле (`docker-compose.yml`) и управлять ей одной командой.

Аналогия: дирижёр и оркестр

Представьте симфонический оркестр. В нём играют скрипки, виолончели, трубы, ударные. Каждый музыкант — это контейнер. Если бы дирижёр подходил к каждому музыканту отдельно и говорил: «Ты начни играть в 14:00, ты — в 14:05, ты — в тональности ми-минор» — это был бы хаос.

Вместо этого дирижёр держит **партитуру** — один лист, где написано, кто, когда и что играет. Он машет палочкой — и весь оркестр начинает одновременно, слаженно, в нужной гармонии.

`docker-compose.yml` — это **партитура вашей системы**. А команда `docker compose up` — это взмах палочки дирижёра.

5.2. Что такое YAML: язык файла docker-compose.yml

Прежде чем разбирать структуру, нужно понять формат. YAML (Yet Another Markup Language) — это формат записи данных, похожий на текстовый список с отступами.

Главное правило YAML: Отступы имеют значение. Используйте **два пробела** (не табуляцию!) для каждого уровня вложенности.

Пример YAML:

```
In [2]: name: Иван
        age: 30
```

```
skills:
  - Python
  - Docker
  - SQL
address:
  city: Москва
  street: Ленина
```

Разбор:

- `name: Иван` — ключ и значение (строка).
- `age: 30` — ключ и число.
- `skills:` — ключ, под которым **список** (начинается с `-`).
- `address:` — ключ, под которым **вложенный объект** (с отступом).

В `docker-compose.yml` мы используем тот же принцип: ключи, значения, списки, вложенности.

5.3. Структура `docker-compose.yml`: сверху вниз

Создайте рабочую папку:

```
In [3]: mkdir ~/docker-module5
        cd ~/docker-module5
```

Внутри создайте файл `docker-compose.yml`. Рассмотрим его структуру по блокам.

5.3.1. Версия схемы (`version`) — устарело, но встречается

В старых версиях Compose первой строкой шла:

```
In [4]: version: "3.9"
```

В современном Docker Compose (Plugin, начиная с 2021 года) это **не обязательно**. Мы будем писать файл без `version`, в формате Compose Specification.

5.3.2. Сервисы (`services`) — сердце файла

Сервис — это описание одного контейнера (или группы контейнеров). Внутри `services` перечисляются все «музыканты оркестра».

```
In [5]: services:
        api:
            # описание API-контейнера
        db:
            # описание базы данных
        redis:
            # описание брокера
```

Имя сервиса (`api` , `db` , `redis`) — это не просто ярлык. Это **DNS-имя**, по которому другие контейнеры будут находить этот сервис в сети. Мы разберём это подробнее в разделе о сетях.

5.3.3. Инструкция `image` — использовать готовый образ

Если сервис должен использовать готовый образ из Docker Hub (например, PostgreSQL или Redis):

```
In [6]: services:
        db:
            image: postgres:15-alpine
        redis:
            image: redis:7-alpine
```

Docker скачает эти образы автоматически при первом запуске.

5.3.4. Инструкция `build` — собрать образ из Dockerfile

Если сервис — это ваше собственное приложение, и рядом лежит `Dockerfile` :

```
In [7]: services:
        api:
            build:
                context: .          # где искать Dockerfile (текущая папка)
                dockerfile: Dockerfile # имя файла (можно не указывать, если Dockerfile)
```

Или короткая форма:

```
In [8]: services:
        api:
            build: .
```

При `docker compose up` Docker автоматически соберёт образ для сервиса `api`.

5.3.5. Инструкция `ports` — проброс портов

Синтаксис: `"ХОСТ:КОНТЕЙНЕР"`.

```
In [9]: services:
        api:
          ports:
            - "8000:8000"
```

Только сервисы, к которым нужен доступ **извне** (из браузера, из Postman), нуждаются в `ports`. База данных и Redis обычно не пробрасываются наружу — они общаются только внутри сети контейнеров.

5.3.6. Инструкция `environment` — переменные окружения

Способ 1: список строк `"КЛЮЧ=ЗНАЧЕНИЕ"` :

```
In [10]: services:
         db:
           environment:
             - POSTGRES_USER=admin
             - POSTGRES_PASSWORD=secret123
             - POSTGRES_DB=shop
```

Способ 2: объект (ключ: значение):

```
In [11]: services:
         db:
           environment:
             POSTGRES_USER: admin
             POSTGRES_PASSWORD: secret123
             POSTGRES_DB: shop
```

Оба способа равнозначны. Используйте тот, который вам понятнее.

5.3.7. Инструкция `volumes` — монтирование томов

Том (volume) — это способ хранить данные вне контейнера. Когда контейнер удаляется, данные в томе сохраняются.

В Compose есть два типа `volumes` :

1. Именованные тома (named volumes) — управляются Docker, хранятся в специальной папке Docker'a. Используются для данных БД.

```
In [12]: services:
          db:
            volumes:
              - pg_data:/var/lib/postgresql/data

          volumes:
            pg_data:
```

2. Bind mounts — привязка к конкретной папке на хосте. Используются для кода во время разработки.

```
In [13]: services:
          api:
            volumes:
              - ./src:/app/src
```

5.3.8. Инструкция `depends_on` — порядок запуска

```
In [14]: services:
          api:
            depends_on:
              - db
              - redis
```

Это говорит Compose: «Сначала запусти `db` и `redis` , потом — `api` ».

Важное ограничение: `depends_on` ждёт только **запуска** контейнера, а не готовности сервиса внутри. PostgreSQL может запуститься за 2 секунды, но инициализация БД займёт 5 секунд. В это время API уже попытается подключиться и упадёт. Для production есть более сложные паттерны (healthchecks), но для обучения `depends_on` достаточно.

5.4. Сети в Docker Compose: магия DNS

5.4.1. Как контейнеры находят друг друга

В Модуле 2 мы запускали контейнеры изолированно. Они не знали друг о друге. Чтобы они общались, нужна **общая сеть**.

Docker Compose **автоматически** создаёт для вашего проекта отдельную сеть. Все сервисы из `docker-compose.yml` подключаются к ней.

Ключевая магия: Внутри этой сети контейнеры могут обращаться друг к другу **по имени сервиса**.

Аналогия: общежитие с именами на дверях

Представьте общежитие. В каждой комнате живёт человек. На дверях написаны имена: «Иван», «Мария», «Алексей». Если Иван хочет передать Марии записку, он не спрашивает: «Какой у тебя IP-адрес?» — он просто пишет на конверте «Мария» и кладёт в почтовый ящик. Почтальон (DNS) знает, в какую комнату доставить.

В Docker Compose:

- Комната = контейнер.
- Имя на двери = имя сервиса (`db` , `redis` , `api`).
- Почтальон = встроенный DNS-сервер Docker.

Поэтому ваше API может подключаться к базе данных не по IP (`172.18.0.3`), а просто по имени:

```
In [15]: postgresql://admin:secret@db:5432/shop
```

Здесь `db` — это имя сервиса. Docker автоматически превратит его в IP-адрес контейнера.

5.4.2. Почему это безопасно

Сеть, созданная Compose, **изолирована** от внешнего мира. Контейнеры из других проектов или с хоста не могут просто так подключиться к вашей БД. Только сервисы внутри этого `docker-compose.yml` видят друг друга (если вы явно не открыли порты наружу).

5.5. Volumes: персистентность данных

5.5.1. Проблема: данные исчезают

Контейнеры по умолчанию **временны**. Если вы удалите контейнер PostgreSQL — все базы данных внутри него исчезнут. Это как выкинуть холодильник вместе с продуктами.

5.5.2. Решение: именованные тома

Именованный том — это папка на диске хоста, которую Docker управляет сам. Вы говорите: «Docker, создай папку с именем `pg_data` и подключи её к контейнеру БД». Docker создаёт эту папку в своём хранилище (обычно `/var/lib/docker/volumes/`).

```
In [16]: services:
          db:
            image: postgres:15-alpine
            volumes:
              - pg_data:/var/lib/postgresql/data

          volumes:
            pg_data:
```

Как это работает:

- PostgreSQL пишет данные в `/var/lib/postgresql/data` внутри контейнера.
- Docker перенаправляет эти записи в папку `pg_data` на хосте.
- Если контейнер остановится или удалится — папка `pg_data` останется.
- При следующем запуске новый контейнер PostgreSQL подключит тот же том и увидит все старые данные.

Аналогия: Том — это **сейф в стене**. Холодильник (контейнер) можно выкинуть и купить новый, но сейф остаётся в стене со всем содержимым.

5.6. Практика: собираем связку API + PostgreSQL + Redis

Шаг 1. Структура проекта

В папке `~/docker-module5` создайте такую структуру:

```
In [17]: docker-module5/
          ├── docker-compose.yml
          ├── Dockerfile
          ├── requirements.txt
          ├── .dockerignore
          └── app/
```

```
├─ __init__.py
└─ main.py
```

Создайте папки и файлы:

```
In [18]: mkdir -p app
touch app/__init__.py
touch app/main.py
touch Dockerfile
touch requirements.txt
touch .dockerignore
touch docker-compose.yml
```

Шаг 2. Код приложения

requirements.txt :

```
In [19]: fastapi==0.111.0
uvicorn[standard]==0.30.0
psycpg2-binary==2.9.9
redis==5.0.0
sqlalchemy==2.0.30
```

app/main.py :

```
In [20]: import os
import time
import redis
from fastapi import FastAPI, HTTPException
from sqlalchemy import create_engine, text
from sqlalchemy.exc import OperationalError

app = FastAPI(title="Shop API with DB and Redis")

# --- Подключение к PostgreSQL ---
# DATABASE_URL берётся из переменных окружения
# Формат: postgresql://user:password@host:port/dbname
DATABASE_URL = os.getenv(
    "DATABASE_URL",
    "postgresql://admin:secret@localhost:5432/shop"
)
```

```

# Создаём движок SQLAlchemy
# pool_pre_ping=True проверяет соединение перед использованием
engine = create_engine(DATABASE_URL, pool_pre_ping=True)

# --- Подключение к Redis ---
REDIS_URL = os.getenv("REDIS_URL", "redis://localhost:6379")
redis_client = redis.from_url(REDIS_URL, decode_responses=True)

# --- Эндпоинты ---

@app.get("/")
def read_root():
    return {
        "message": "API работает!",
        "services": {
            "database": "PostgreSQL",
            "broker": "Redis"
        }
    }

@app.get("/health")
def health_check():
    """Проверяем, доступны ли БД и Redis."""
    status = {"api": "ok", "database": "unknown", "redis": "unknown"}

    # Проверка PostgreSQL
    try:
        with engine.connect() as conn:
            conn.execute(text("SELECT 1"))
            status["database"] = "connected"
    except OperationalError as e:
        status["database"] = f"error: {str(e)}"

    # Проверка Redis
    try:
        redis_client.ping()
        status["redis"] = "connected"
    except Exception as e:
        status["redis"] = f"error: {str(e)}"

    # Если что-то не так — возвращаем 503
    if status["database"] != "connected" or status["redis"] != "connected":
        raise HTTPException(status_code=503, detail=status)

    return status

```

```

@app.post("/orders")
def create_order(product: str, quantity: int):
    """Создаём заказ в БД и кладём задачу в Redis."""
    try:
        with engine.connect() as conn:
            # Вставляем заказ
            result = conn.execute(
                text("INSERT INTO orders (product, quantity, status) VALUES (:p,
:p, 'new') RETURNING id"),
                {"p": product, "q": quantity}
            )
            order_id = result.scalar()
            conn.commit()

            # Кладём задачу в Redis (симуляция: обработать заказ)
            redis_client.xadd("orders:stream", {
                "order_id": str(order_id),
                "product": product,
                "quantity": str(quantity),
                "action": "process"
            })

            return {
                "order_id": order_id,
                "product": product,
                "quantity": quantity,
                "status": "created",
                "message": "Заказ создан и поставлен в очередь"
            }
    except OperationalError as e:
        raise HTTPException(status_code=500, detail=f"Database error: {str(e)}")

@app.get("/orders")
def list_orders():
    """Получаем список заказов из БД."""
    try:
        with engine.connect() as conn:
            result = conn.execute(text("SELECT * FROM orders ORDER BY id DESC LIM
IT 10"))
            rows = result.mappings().all()
            return {"orders": [dict(row) for row in rows]}
    except OperationalError as e:
        raise HTTPException(status_code=500, detail=f"Database error: {str(e)}")

@app.get("/queue/info")

```

```

def queue_info():
    """Информация о длине очереди в Redis."""
    try:
        length = redis_client.xlen("orders:stream")
        return {"queue_name": "orders:stream", "length": length}
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"Redis error: {str(e)}")

@app.on_event("startup")
def startup():
    """При старте создаём таблицу заказов, если её нет."""
    time.sleep(2) # Даём БД время на инициализацию
    try:
        with engine.connect() as conn:
            conn.execute(text("""
                CREATE TABLE IF NOT EXISTS orders (
                    id SERIAL PRIMARY KEY,
                    product VARCHAR(255) NOT NULL,
                    quantity INTEGER NOT NULL,
                    status VARCHAR(50) DEFAULT 'new',
                    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
                )
            """))
            conn.commit()
            print("Таблица orders создана или уже существует")
    except OperationalError as e:
        print(f"Не удалось подключиться к БД при старте: {e}")

```

Шаг 3. Dockerfile для API

Dockerfile :

```

In [21]: FROM python:3.11-slim

ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

```

```
EXPOSE 8000
```

```
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Шаг 4. .dockerignore

.dockerignore :

```
In [22]: __pycache__/  
*.pyc  
*.pyo  
*.pyd  
.env  
.venv/  
venv/  
.git/  
.gitignore  
*.md
```

Шаг 5. docker-compose.yml — сердце модуля

docker-compose.yml :

```
In [23]: services:  
    # =====  
    # Сервис 1: База данных PostgreSQL  
    # =====  
    db:  
        image: postgres:15-alpine  
        container_name: shop_db  
        environment:  
            POSTGRES_USER: admin  
            POSTGRES_PASSWORD: secret123  
            POSTGRES_DB: shop  
        volumes:  
            # Именованный том для персистентности данных  
            - pg_data:/var/lib/postgresql/data  
            # Порт НЕ пробрасываем наружу (только внутри сети Compose)  
            # Если нужен доступ с хоста для отладки, раскомментируйте:  
            # ports:  
            #   - "5432:5432"  
  
    # =====
```

```

# Сервис 2: Брокер сообщений Redis
# =====
redis:
  image: redis:7-alpine
  container_name: shop_redis
  # Redis хранит данные в памяти, но можно включить персистентность
  command: redis-server --appendonly yes
  volumes:
    - redis_data:/data

# =====
# Сервис 3: API на FastAPI
# =====
api:
  build:
    context: .
    dockerfile: Dockerfile
  container_name: shop_api
  ports:
    # Пробрасываем порт 8000 наружу, чтобы был доступ из браузера
    - "8000:8000"
  environment:
    # Используем ИМЕНА СЕРВИСОВ как хосты!
    # db — это имя сервиса выше. Docker сам подставит IP-адрес.
    DATABASE_URL: postgresql://admin:secret123@db:5432/shop
    REDIS_URL: redis://redis:6379
  depends_on:
    # Сначала запустить db и redis, потом api
    - db
    - redis

# =====
# Объявление именованных томов
# =====
volumes:
  pg_data:
    # Docker создаст и управляет этим томом автоматически
  redis_data:

```

Разбор ключевых строк:

db:

- `image: postgres:15-alpine` — готовый образ PostgreSQL, лёгкий вариант на Alpine Linux.

- `environment:` — задаём логин, пароль и имя базы. PostgreSQL при первом запуске создаст БД `shop` и пользователя `admin`.
- `volumes:` - `pg_data:/var/lib/postgresql/data` — данные БД сохраняются в том `pg_data`. Если остановить и снова запустить проект — таблицы и заказы останутся.

`redis:`

- `image: redis:7-alpine` — готовый образ Redis.
- `command: redis-server --appendonly yes` — включаем режим AOF (Append Only File), чтобы Redis сохранял данные на диск. Это введение в персистентность Redis.
- `volumes:` - `redis_data:/data` — файл `appendonly.aof` будет храниться в этом томе.

`api:`

- `build:` — собираем образ из Dockerfile в текущей папке.
- `ports:` - `"8000:8000"` — открываем API наружу. Только этот сервис доступен с хоста.
- `environment:` — передаём URL для подключения. Обратите внимание: `db` и `redis` — это имена сервисов, не IP-адреса!
- `depends_on:` — гарантируем порядок запуска.

`volumes:` **внизу файла:**

Это **объявление** томов. До этого мы их только **использовали** в сервисах. Здесь мы говорим Compose: «Эти тома существуют, управляй ими».

5.7. Запуск и управление системой

5.7.1. Первая команда: поднять всё

In [24]: `docker compose up`

Что происходит:

1. Docker создаёт сеть для проекта (автоматически, имя будет `docker-module5_default`).
2. Создаёт тома `pg_data` и `redis_data` (если их ещё нет).
3. Скачивает образы `postgres:15-alpine` и `redis:7-alpine` (если их нет локально).
4. Собирает образ для `api` (если Dockerfile изменился).
5. Запускает контейнеры в порядке `depends_on`.
6. Показывает логи всех трёх сервисов в одном терминале (цветом различаются).

Вы увидите что-то вроде:

```
In [25]:  [+] Running 4/4
          ✓ Network docker-module5_default      Created
          ✓ Volume "docker-module5_pg_data"     Created
          ✓ Volume "docker-module5_redis_data"  Created
          ✓ Container shop_db                   Started
          ✓ Container shop_redis                Started
          ✓ Container shop_api                  Started
Attaching to shop_api, shop_db, shop_redis
shop_db   | The files belonging to this database system will be owned by user "p
shop_db   | ostgres".
shop_db   | This user must also own the server process.
...
shop_api  | INFO:      Started server process [1]
shop_api  | INFO:      Waiting for application startup.
shop_api  | Таблица orders создана или уже существует
shop_api  | INFO:      Application startup complete.
```

5.7.2. Фоновый запуск

Чтобы освободить терминал:

```
In [26]:  docker compose up -d
```

Флаг `-d` (detached) — запуск в фоне.

5.7.3. Проверка работы

Откройте браузер или используйте `curl` :

```
In [27]:  # Проверка API
          curl http://localhost:8000/

          # Проверка здоровья (должно показать, что БД и Redis подключены)
          curl http://localhost:8000/health

          # Создаём заказ
          curl -X POST "http://localhost:8000/orders?product=Ноутбук&quantity=2"

          # Смотрим список заказов
```

```
curl http://localhost:8000/orders

# Смотрим информацию об очереди Redis
curl http://localhost:8000/queue/info
```

5.7.4. Просмотр логов

```
In [28]: # Логи всех сервисов
docker compose logs

# Логи конкретного сервиса
docker compose logs api
docker compose logs db
docker compose logs redis

# Логи в реальном времени (follow)
docker compose logs -f api
```

5.7.5. Выполнение команд внутри контейнеров

```
In [29]: # Подключиться к БД через psql
docker compose exec db psql -U admin -d shop

# Внутри psql можно выполнить:
# \dt                — список таблиц
# SELECT * FROM orders; — посмотреть заказы
# \q                 — выйти

# Подключиться к Redis через redis-cli
docker compose exec redis redis-cli

# Внутри redis-cli:
# KEYS *              — список ключей
# XLEN orders:stream — длина потока
# XREAD COUNT 10 STREAMS orders:stream 0 — прочитать сообщения
# QUIT               — выйти
```

5.7.6. Перезапуск отдельного сервиса

Если вы изменили код в `app/main.py` :

```
In [30]: # Пересобрать и перезапустить только API
docker compose up --build api
```

5.7.7. Остановка и удаление

```
In [31]: # Остановить все контейнеры (данные в volumes сохранятся)
docker compose down

# Остановить и удалить ВСЁ: контейнеры, сеть, тома (данные пропадут!)
docker compose down -v
```

Будьте осторожны с `-v` ! Это удалит `pg_data` и `redis_data` . Все заказы исчезнут.

5.8. Что происходит под капотом: детальный разбор

5.8.1. Автоматическая сеть

Когда вы выполняете `docker compose up` , Compose создаёт сеть типа `bridge` . Внутри неё каждый контейнер получает IP-адрес (например, `172.20.0.2` , `172.20.0.3`).

Но вам не нужно запоминать эти IP! Встроенный DNS-сервер Docker отвечает на запросы по имени сервиса. Когда ваше API делает подключение к `db:5432` , происходит следующее:

1. Библиотека `psycopg2` просит операционную систему разрешить имя `db` .
2. ОС контейнера `api` спрашивает Docker DNS: «Кто такой db?»
3. Docker DNS отвечает: «Это контейнер с IP `172.20.0.2` ».
4. Соединение устанавливается.

5.8.2. Изоляция по умолчанию

Порты, которые не проброшены через `ports` , доступны **только внутри сети Compose**. С вашего компьютера (хоста) вы не сможете подключиться к PostgreSQL напрямую:

```
In [32]: # Это НЕ сработает (если вы не пробросили порт 5432)
psql -h localhost -p 5432 -U admin -d shop
```

Это хорошо с точки зрения безопасности. База данных «спрятана» внутри сети, к ней имеет доступ только API.

5.8.3. Тома и их место на диске

Docker хранит именованные тома в своей папке (обычно `/var/lib/docker/volumes/` на Linux, внутри виртуальной машины Docker Desktop на Windows/macOS). Вы можете посмотреть список:

```
In [33]: docker volume ls
```

Вы увидите:

```
In [34]: local      docker-module5_pg_data
         local      docker-module5_redis_data
```

Эти тома живут независимо от контейнеров. Даже после `docker compose down` они остаются. Их можно удалить только явно: `docker compose down -v` или `docker volume rm`.

5.9. Итоги модуля: чек-лист

- ☐ Понимаю, зачем нужен Docker Compose: оркестрация множества контейнеров одной командой.
- ☐ Знаю структуру `docker-compose.yml`: `services`, `volumes`, `networks` (автоматически).
- ☐ Умею использовать `image` для готовых образов и `build` для своих.
- ☐ Знаю, как пробрасывать порты (`ports`) и зачем это нужно только для «внешних» сервисов.
- ☐ Умею передавать переменные окружения через `environment`.
- ☐ Понимаю, как работает **DNS внутри Compose**: контейнеры находят друг друга по **имени сервиса**.
- ☐ Знаю, что `depends_on` задаёт порядок запуска, но не гарантирует готовности сервиса.
- ☐ Понимаю разницу между **именованными томами** (для данных БД) и **bind mounts** (для кода).
- ☐ Умею запускать систему (`docker compose up -d`), смотреть логи (`docker compose logs`), выполнять команды внутри (`docker compose exec`).
- ☐ Умею останавливать систему (`docker compose down`) и понимаю разницу между обычной остановкой и удалением с томами (`-v`).
- ☐ Собрал и запустил связку из трёх сервисов (API + PostgreSQL + Redis), проверил их взаимодействие через HTTP-запросы.

В следующем модуле мы войдём в мир **брокеров сообщений** и паттерна **Producer/Consumer**. Мы поймём, зачем API не должен выполнять тяжёлые задачи самостоятельно, и научимся отдавать работу фоновым процессам через очереди.